Talk to Your Plant

Domain brief: multimodal GraphRAG brain for an EV manufacturing digital twin

Working reference for the hackathon build. Written 2026-10-10. Items marked **[checked]** were looked up while writing this; items marked **[unchecked]** come from general knowledge and need a quick confirm before you rely on them.

1. TL;DR

- **No ready-made 'EV plant' dataset exists.** Build a **synthetic plant backbone** (people, machines, parts, vehicles, orders) that we control, then attach **real public datasets** for sensor, image and text realism.
- **Recommended multimodal pattern: graph-anchored.** Every text chunk, image and sensor window is a node linked to a plant entity (Machine, Vehicle, Part, Worker). Retrieval = find entities, walk the graph, pull evidence of any modality.
- **Do not embed raw time series.** Turn sensor streams into windows, features, anomaly and RUL (remaining useful life) events, and store those as graph nodes. Keep raw data in a time-series store.
- **Real-time = replayed and simulated.** A simulator replays public datasets in accelerated time and emits events on a stream. Your friend's twin and KPI dashboard consume the same stream.
- **Showcase = 6 short user stories** (section 8), each ending in a visible 'why' answer that crosses at least two modalities.

2. Problem framing

The system answers questions such as: how far along is vehicle X, what is blocking it, which machine is about to fail, what to reorder, who is overworked, and what happens if a supplier slips. These need **multi-hop reasoning across connected entities** plus **live state**, which is why a graph beats flat vector search here.

Timeline note. You said a car takes about two months. Physical assembly on a line is usually hours to days; two months fits an **order-to-delivery pipeline** (parts lead time, sub-assembly, queueing, QA, shipping). [unchecked] Model the vehicle as a pipeline of stages with durations, so the progress percentage and delay reasons come out naturally.

3. Multimodal RAG options

Approach	How it works	Pros	Cons	Verdict
A. Text-ify everything	Caption images, summarise sensor windows to text, embed all with one text model.	Simplest. One index. Fast to ship.	Loses image detail. Captions cost LLM calls up front.	Good baseline. Do this first.
B. Shared multimodal embedding	A model that embeds image and text into one space (CLIP-style). Query text finds images directly.	True cross-modal search. 'Find defects like this photo'.	Needs model choice and tuning. Weak on domain jargon.	Add for images only.
C. Per-modality stores + router	Vector DB for text, image index, TSDB for sensors. A router picks the store.	Each store is best at its job.	Joins across stores are manual. Answers feel disconnected.	Use as storage layer under D.
D. Graph-anchored (recommended)	Entities form the graph. Evidence nodes of every modality hang off entities. Query = entity link, graph walk, fetch evidence.	Multi-hop answers. Explains 'why'. Lower latency than whole-corpus graph summarisation because queries start from known entities.	Needs an ontology up front. Entity linking must be good.	**Pick this.**

Framework choice. Microsoft-style GraphRAG builds community summaries over a text corpus; one benchmark found its latency unsuitable for real-time use, and LightRAG came out lighter. [checked, single source, different setups, not a controlled comparison]. Our data is mostly **structured and entity-rich**, so we do not need LLM graph extraction for the backbone; we write the graph directly and use LLM extraction only for unstructured documents (SOPs, incident reports).

Suggested stack (my judgment, not benchmarked): **Neo4j** with its built-in vector index for graph plus embeddings; **TimescaleDB or DuckDB** for sensor history; **Python** for ingestion and agent. Verify current Neo4j licensing and any embedded alternatives before committing.

4. Plant ontology (what the graph holds)

These are the 'concrete information labels'. Every node type below needs an ID, a name, a status and timestamps; the extra properties are what makes questions answerable.

Node	Key properties	Why it exists
Vehicle	vin, model, order_id, target_date, stage, pct_complete, status	The thing being built; progress and delay target.
BuildStage	name (stamping, body-in-white, paint, battery pack, drive unit, final assembly, EOL test, QA), planned_hours, weight, state	Progress % = sum of weights of completed stages (partial credit for in-progress).
Machine	id, type, line, install_date, health_score, wear_pct, rul_hours, last_service, status	Failure prediction, downtime blame.
Sensor / Reading window	sensor_id, kind (vibration, temp, current, pressure), window_start, features, anomaly_score	Time series becomes queryable events.
Part / Component	sku, name, bom_level, supplier, unit_cost, lead_time_days	BOM tree for the vehicle and what stage consumes it.
InventoryLot	sku, qty_on_hand, reorder_point, safety_stock, days_of_cover, location	Low-stock and reorder logic.
Supplier / PurchaseOrder	supplier_id, on_time_rate, order_id, eta, status, qty	Sourcing and delay propagation.
Worker	id, role, skills, shift, hours_this_week, overtime_hours, station	Workload, fatigue, staffing.
Task / WorkOrder	id, type (production, maintenance), assigned_to, machine, vehicle, due, state	Links people, machines, vehicles.
Document	doc_id, kind (SOP, manual, incident, maintenance log), chunks, embedding	Text evidence.
Image / Inspection	image_id, station, vehicle, defect_label, score, embedding	Visual evidence (defects, photos).
Incident / Alert	id, severity, root_cause, resolved_at	Past cases for 'what happened last time'.

Core relationships: Vehicle -[:IN_STAGE]-> BuildStage; BuildStage -[:RUNS_ON]-> Machine; BuildStage -[:CONSUMES]-> Part; Part -[:SUPPLIED_BY]-> Supplier; Part -[:STOCKED_AS]-> InventoryLot; Machine -[:HAS_SENSOR]-> Sensor; Sensor -[:EMITTED]-> Reading; Worker -[:ASSIGNED]-> Task; Task -[:ON]-> Machine/Vehicle; Document/Image/Incident -[:ABOUT]-> any entity.

Progress formula (starting point): pct_complete = sum(weight of finished stages) + weight of current stage x (elapsed / planned). **Delay cause** = first upstream blocker found by walking CONSUMES / RUNS_ON / ASSIGNED edges from the blocked stage and checking part shortages, machine alerts, then staffing.

5. Retrieval flow

- **1. Classify** the question: status lookup, diagnosis ('why'), prediction, recommendation, or what-if.
- **2. Link entities**: map 'VIN 0412', 'the paint booth', 'battery cells' to node IDs.

- **3. Query the graph**: structured questions via text-to-Cypher with a fixed, validated query set; 'why' questions via bounded multi-hop traversal.
- **4. Pull evidence**: vector search restricted to chunks and images attached to the entities found (small, fast, relevant).
- **5. Read live state**: latest readings and anomaly events from the time-series store.
- **6. Answer with citations** to nodes, documents, images and sensor windows so the demo shows its reasoning.

Latency tip: restricting vector search to entity-attached evidence and caching the last-known state of each entity should keep responses interactive. Measure early; do not assume.

6. Data sources

Strategy: **synthetic backbone for structure and text, public datasets for signal realism.** Public sensor sets are not from EV plants, so we relabel them (for example, treat a bearing vibration set as a conveyor or spindle motor). Say this openly in the demo; it is a stand-in, not a claim.

6.1 Sensors and time series

Source	What it is	Use in our twin	Status
NASA C-MAPSS turbofan degradation	Run-to-failure multivariate sensor data, 4 subsets (FD001-FD004), with RUL labels in mirrors.	RUL and wear curves for any rotating machine. Best for 'which machine fails next'.	[checked] Data zip hosted by NASA PCoE (PHM mirror). The simulator itself is listed as unavailable; use the data only.
AI4I 2020 predictive maintenance	10,000 synthetic rows, 14 features, binary failure plus 5 failure modes, ~3.4% failures.	Quick tabular failure model; labels for failure modes (tool wear, heat, power, overstrain).	[checked] UCI 601, CC BY 4.0, loadable with ucimlrepo.
NASA Li-ion battery aging	Charge, discharge, impedance cycles to 30% capacity fade; MATLAB structs.	EV flavour: battery pack test station, cell capacity fade at end-of-line testing.	[checked] NASA open data.
CWRU / Paderborn bearing vibration	Bearing fault vibration signals, 12/48 kHz (CWRU), .mat files.	Vibration spikes on conveyors, motors, spindles. Good for a visual spectrum demo.	[checked partially] Datasets exist; official download pages not confirmed. Check the official sites; avoid unofficial drive mirrors.
Bosch production line (Kaggle 2016)	Anonymised sensor features per part through a production line, with pass/fail.	Line-level quality: which station correlates with failures.	[unchecked] Verify access and rules on Kaggle.
SECOM (UCI)	Semiconductor process sensors with pass/fail.	Backup for process-quality signals.	[unchecked]

6.2 Images (visual inspection)

Source	What it is	Use in our twin	Status
MVTec AD	5,000+ high-res images, 15 object and texture categories, 70+ defect types, pixel masks, defect-free training sets.	Defect photos at QA stations; 'find similar past defects' via image embeddings.	[checked] MVTec. Check its license terms (it is non-commercial as far as I recall [unchecked]).
Severstal steel defects	Steel sheet surface images with 4 defect classes, RLE masks.	Stamping / body panel defect inspection.	[checked] Kaggle competition data; redistribution is restricted, so keep it local and read the Rules tab.
Generated images	Synthetic photos or diagrams of stations, parts, wiring (via an image model).	Fill gaps: station photos, part images, labelled diagrams.	Do last; label as synthetic.

6.3 Text and tabular business data

Source	What it is	Use in our twin	Status
DataCo Smart Supply Chain	Orders, shipments, delivery performance, profit; plus clickstream logs. Roughly 180k order rows.	Pattern for orders, lead times, late deliveries; adapt to parts and suppliers.	[checked] Kaggle and Mendeley. Retail-flavoured, so re-map columns.

Source	What it is	Use in our twin	Status
LLM-generated SOPs, manuals, maintenance logs, shift reports, incident write-ups	Plausible documents per machine and process, written to our ontology.	The text corpus. Gives the RAG real documents to cite.	Generate in bulk, seed with realistic failure stories that match sensor anomalies.
Public EV and auto knowledge	OEM sustainability and manufacturing reports, EV teardown articles, standards summaries, NHTSA recall and complaint records.	Domain grounding: real stage names, part lists, common failure modes.	[unchecked] Confirm each source's terms before ingesting.
Synthetic ERP tables	Workers, shifts, BOM, inventory, POs, work orders (generated with fixed random seeds).	The backbone. Everything else attaches here.	We generate. Cheap and fully controllable.

Gap to close: true EV-plant BOMs and line data are proprietary. Invent a believable BOM (battery cells -> modules -> pack; motor; inverter; chassis; body panels; paint; interior) and document it in DECISIONS.md so the team shares one source of truth.

7. Real-time layer

- **Simulator** (Python): replays sensor datasets at 10x to 100x speed, advances vehicles through stages, consumes inventory, rolls worker hours, injects scripted faults on demand ('stuck welder', 'supplier delay').
- **Transport:** start with an in-process queue or Redis Streams; move to MQTT or Kafka only if time allows. The United Manufacturing Hub project ships simulated PackML, OPC UA and MQTT feeds if you want an off-the-shelf industrial protocol demo. [checked, UMH]
- **Event contract** (agree with your friend now): `{event_id, ts, type, entity_type, entity_id, payload}`. Types: reading, stage_change, stock_change, po_update, shift_update, alert.
- **Two consumers, one stream:** (1) brain ingestion updates graph and TSDB; (2) twin and KPI dashboard render the same events.
- **Replayable scenarios:** store a scripted timeline per user story so the demo is deterministic.

8. Showcase user stories

#	Persona	Question	What the brain does	Modalities
1	Plant manager	'Why is VIN 0412 three days late?'	Vehicle -> stuck stage -> consumed part short -> supplier PO late; also welder down on the same stage. Ranks causes.	Graph, inventory, text log
2	Maintenance lead	'Which machines will fail this week and what do I do?'	RUL from sensor windows, rank by risk and downstream vehicles affected, retrieve matching SOP and past incident.	Time series, text
3	Inventory keeper	'What runs out before Friday and what should I order?'	Days of cover per SKU vs consumption forecast and supplier lead time; draft PO with supplier on-time rate.	Tabular, graph
4	Shift supervisor	'Who is overworked and who can cover?'	Overtime hours vs limits, skills match, station coverage, suggested swap.	Tabular, graph
5	QA engineer	Uploads a photo of a paint defect: 'Seen this before?'	Image embedding finds similar past defects, links to booth, shift and temperature drift readings, returns probable root cause.	Image, time series, text
6	Manager (what-if)	'What if the cell supplier slips 10 days?'	Propagates delay through BOM and stages, shows affected vehicles and revenue at risk, proposes mitigation.	Graph, simulation

Demo arc suggestion: open on story 1 (the 'why' crossing inventory, machine and supplier), then 5 (image upload for the visual wow), end on 6 (what-if) which hands over to your friend's simulator and KPI view.

9. Handoff to the digital twin and KPI dashboard

KPI	Definition	Fed by
Build completion %	Weighted stage progress per vehicle and per line.	stage_change events
OEE	Availability x performance x quality per machine.	Machine status, cycle counts, defects
First-pass yield / scrap rate	Share of units passing QA first time.	Inspection events
MTBF / MTTR	Mean time between failures / to repair.	Alert and work-order events
Days of cover	On-hand / daily consumption per SKU.	stock_change events
On-time supplier rate	POs delivered by ETA.	po_update events
Overtime hours	Hours beyond limit per worker and team.	shift_update events
Order-to-delivery time	Order date to completion date.	Vehicle lifecycle

10. Suggested build order

- **Step 1.** Fix the ontology and event schema (this document, section 4 and 7). Log in DECISIONS.md.
- **Step 2.** Generate the synthetic backbone and load the graph. Answer story 1 and 3 from structured data only.
- **Step 3.** Add the simulator and streaming ingestion; make state live.
- **Step 4.** Add sensors: pick C-MAPSS or AI4I, build RUL / anomaly events, answer story 2.
- **Step 5.** Add documents: generate SOPs and logs, embed, attach to entities.
- **Step 6.** Add images (MVTec subset), answer story 5.
- **Step 7.** What-if propagation (story 6) and polish citations.

Open questions

- LLM and embedding model choice (and whether the hackathon allows hosted APIs).
- Scale of the synthetic plant: how many lines, machines, vehicles, workers.
- Who owns the simulator: this repo or your friend's twin side.
- Neo4j vs another graph store given hackathon deploy constraints.

Sources consulted

https://data.phmsociety.org/nasa/

https://archive.ics.uci.edu/dataset/601

https://data.nasa.gov/dataset/li-ion-battery-aging-datasets

https://www.mvtec.com/company/research/datasets/mvtec-ad

https://data.mendeley.com/datasets/8gx2fvg2k6/1

https://learn.umh.app/topic/opc-ua-simulator

https://github.com/VictorBauler/awesome-bearing-dataset